

CHAPTER 12

GRAPHS

IN THIS CHAPTER, YOU WILL:

- Learn about graphs
- Become familiar with the basic terminology of graph theory
- Discover how to represent graphs in computer memory
- Examine and implement various graph traversal algorithms
- Learn how to implement a shortest path algorithm
- Examine and implement the minimum spanning tree algorithm
- Explore topological sort
- Learn how to find Euler circuits in a graph

In previous chapters, you learned various ways to represent and manipulate data. This chapter discusses how to implement and manipulate graphs, which have numerous applications in computer science.

Introduction

In 1736, the following problem was posed. In the town of Königsberg (now called Kaliningrad), the river Pregel (Pregolya) flows around the island Kneiphof and then divides into two. See Figure 12-1.

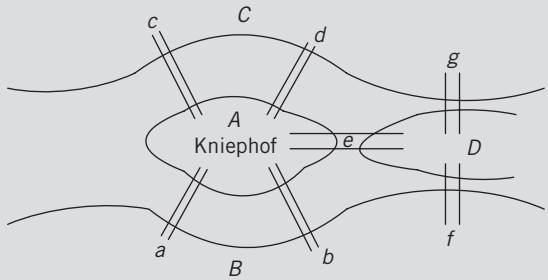


FIGURE 12-1 The Königsberg bridge problem

The river has four land areas (*A*, *B*, *C*, *D*), as shown in the figure. These land areas are connected using seven bridges, as shown in Figure 12-1. The bridges are labeled *a*, *b*, *c*, *d*, *e*, *f*, and *g*. The Königsberg bridge problem is as follows: Starting at one land area, is it possible to walk across all the bridges exactly once and return to the starting land area? In 1736, Euler represented the Königsberg bridge problem as a graph, as shown in Figure 12-2, and answered the question in the negative. This marked (as recorded) the birth of graph theory.

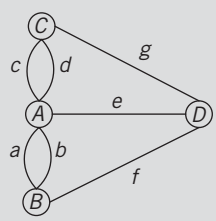


FIGURE 12-2 Graph representation of the Königsberg bridge problem

Over the past 200 years, graph theory has been applied to a variety of applications. Graphs are used to model electrical circuits, chemical compounds, highway maps, and so on. They are also used in the analysis of electrical circuits, finding the shortest route, project planning, linguistics, genetics, social science, and so forth. In this chapter, you learn about graphs and their applications in computer science.

Graph Definitions and Notations

To facilitate and simplify our discussion, we borrow a few definitions and terminology from set theory. Let X be a set. If a is an element of X, we write $a \in X$. (The symbol "$\in$" means "belongs to.") A set Y is called a **subset** of X if every element of Y is also an element of X. If Y is a subset of X, we write $Y \subseteq X$. (The symbol "$\subseteq$" means "is a subset of.") The **intersection** of sets A and B, written $A \cap B$, is the set of all elements that are in A and B; that is, $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$. (The symbol "$\cap$" means "intersection.") The **union** of sets A and B, written $A \cup B$, is the set of all elements that are in A or in B; that is, $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$. (The symbol "$\cup$" means "union.") For sets A and B, the set $A \times B$ is the set of all ordered pairs of elements of A and B; that is, $A \times B = \{(a, b) \mid a \in A, b \in B\}$. (The symbol "$\times$" means "**Cartesian product**.")

A **graph** G is a pair, $G = (V, E)$, where V is a finite nonempty set, called the set of **vertices** of G and $E \subseteq V \times V$. That is, the elements of E are pairs of elements of V. E is called the set of **edges** of G. G is called **trivial** if it has only one vertex.

Let $V(G)$ denote the set of vertices, and $E(G)$ denote the set of edges of a graph G. If the elements of E are ordered pairs, G is called a **directed graph** or **digraph**; otherwise, G is called an **undirected graph**. In an undirected graph, the pairs (u, v) and (v, u) represent the same edge.

Let G be a graph. A graph H is called a **subgraph** of G if $V(H) \subseteq V(G)$ and $E(H) \subseteq E(G)$; that is, every vertex of H is a vertex of G, and every edge in H is an edge in G.

A graph can be shown pictorially. The vertices are drawn as circles, and a label inside the circle represents the vertex. In an undirected graph, the edges are drawn using lines. In a directed graph, the edges are drawn using arrows.

EXAMPLE 12-1

Figure 12-3 shows some examples of undirected graphs.

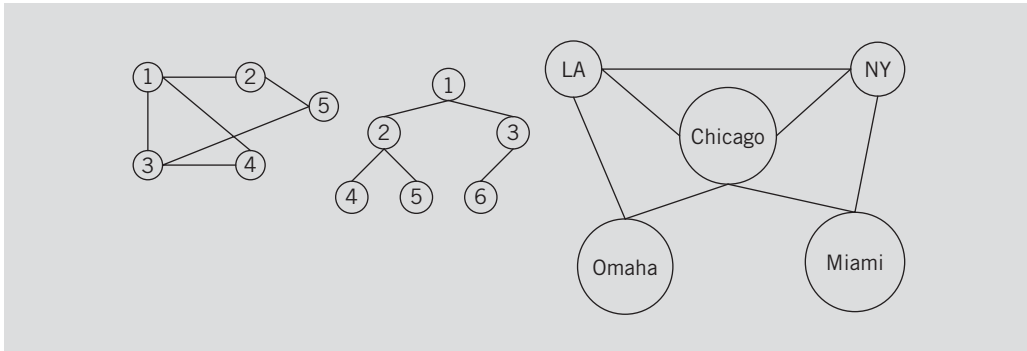


FIGURE 12-3 Various undirected graphs

EXAMPLE 12-2

Figure 12-4 shows some examples of directed graphs.

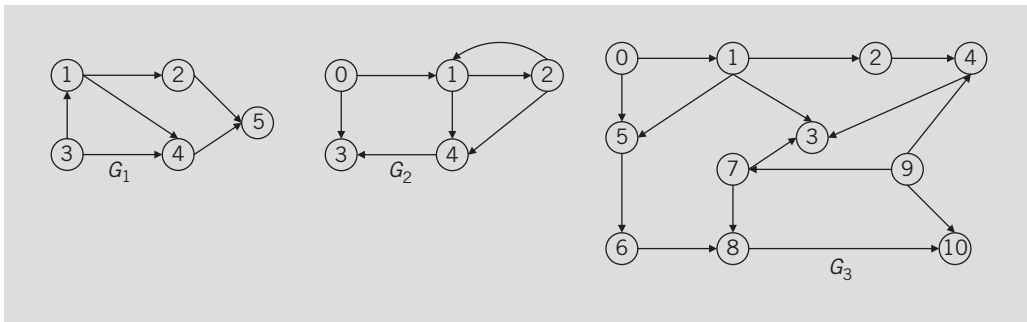


FIGURE 12-4 Various directed graphs

For the graphs of Figure 12-4, we have

$V(G_1) = \{1, 2, 3, 4, 5\}$
$V(G_2) = \{0, 1, 2, 3, 4\}$
$V(G_3) = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

$E(G_1) = \{(1, 2), (1, 4), (2, 5), (3, 1), (3, 4), (4, 5)\}$
$E(G_2) = \{(0, 1), (0, 3), (1, 2), (1, 4), (2, 1), (2, 4), (4, 3)\}$
$E(G_3) = \{(0, 1), (0, 5), (1, 2), (1, 3), (1, 5), (2, 4), (4, 3), (5, 6), (6, 8), (7, 3), (7, 8), (8, 10), (9, 4), (9, 7), (9, 10)\}$

Let G be an undirected graph. Let u and v be two vertices in G. Then u and v are called **adjacent** if there is an edge from one to the other; that is, $(u, v) \in E$. An edge incident on a single vertex is called a **loop**. If two edges, e_1 and e_2, are associated with the same pair of vertices $\{u, v\}$, then e_1 and e_2 are called **parallel edges**. A graph is called a **simple graph** if it has no loops and no parallel edges. Let $e = (u, v)$ be an edge in G. We then say that edge e is **incident** on the vertices u and v. The degree of u, written deg(u) or $d(u)$, is the number of edges incident with u. We make the convention that each loop on a vertex u contributes 2 to the degree of u. u is called an **even (odd) degree** vertex if the degree of u is even (odd). There is a **path** from u to v if there is a sequence of vertices $u_1, u_2, \ldots, u_n$ such that $u = u_1$, $u_n = v$ and (u_i, u_{i+1}) is an edge for all $i = 1, 2, \ldots, n - 1$. Vertices u and v are called **connected** if there is a path from u to v. A **simple path** is a path in which all the vertices, except possibly the first and last vertices, are distinct. A **cycle** in G is a simple path in which the first and last vertices are the same. G is called **connected** if there is a path from any vertex to any other vertex. A maximal subset of connected vertices is called a **component** of G.

Let G be a directed graph, and let u and v be two vertices in G. If there is an edge from u to v, that is, $(u, v) \in E$, we say that u is **adjacent to** v and v is **adjacent from** u. The definitions of the paths and cycles in G are similar to those for undirected graphs. G is called **strongly connected** if any two vertices in G are connected.

Consider the directed graphs of Figure 12-4. In G_1, `1-4-5` is a path from vertex `1` to vertex `5`. There are no cycles in G_1. In G_2, `1-2-1` is a cycle. In G_3, `0-1-2-4-3` is a path from vertex `0` to vertex `3`; `1-5-6-8-10` is a path from vertex `1` to vertex `10`. There are no cycles in G_3.

Graph Representation

To write programs that process and manipulate graphs, the graphs must be stored—that is, represented—in computer memory. A graph can be represented (in computer memory) in several ways. We now discuss two commonly used ways: adjacency matrices and adjacency lists.

Adjacency Matrices

Let G be a graph with n vertices, where $n > 0$. Let $V(G) = \{v_1, v_2, \ldots, v_n\}$. The adjacency matrix A_G is a two-dimensional $n \times n$ matrix such that the (i, j)th entry of A_G is 1 if there is an edge from v_i to v_j; otherwise, the (i, j)th entry is 0. That is,

$$A_G(i,j) = \begin{cases} 1 & \text{if } (v_i, v_j) \in E(G) \\ 0 & \text{otherwise} \end{cases}$$

In an undirected graph, if $(v_i, v_j) \in E(G)$, then $(v_j, v_i) \in E(G)$, so $A_G(i, j) = 1 = A_G(j, i)$. It follows that the adjacency matrix of an undirected graph is symmetric.

EXAMPLE 12-3

Consider the directed graphs of Figure 12-4. The adjacency matrices of the directed graphs G_1 and G_2 are as follows:

$$A_{G_1} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad A_{G_2} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}.$$

Adjacency Lists

Let G be a graph with n vertices, where $n > 0$. Let $V(G) = \{v_1, v_2, \ldots, v_n\}$. In the adjacency list representation, corresponding to each vertex, v, there is a linked list such that each node of the linked list contains the vertex, u, such that $(v, u) \in E(G)$. Because there are n nodes, we use an array, A, of size n, such that $A[i]$ is a reference variable pointing to the first node of the linked list containing the vertices to which v_i is adjacent. Clearly, each node has two components, say `vertex` and `link`. The component `vertex` contains the index of the vertex adjacent to vertex i.

EXAMPLE 12-4

Consider the directed graphs of Figure 12-4. Figure 12-5 shows the adjacency list of the directed graph G_2.

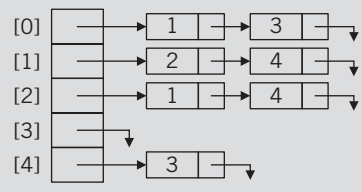


FIGURE 12-5 Adjacency list of graph G_2 of Figure 12-4

Figure 12-6 shows the adjacency list of the directed graph G_3.

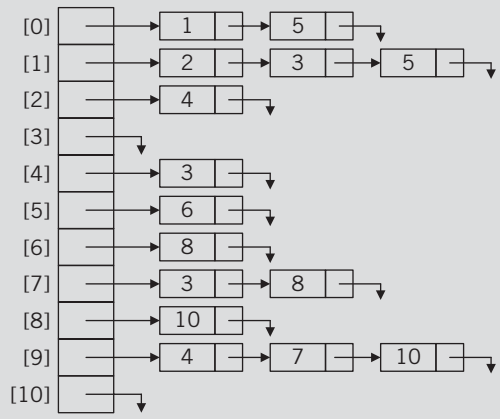


FIGURE 12-6 Adjacency list of graph G_3 of Figure 12-4

Operations on Graphs

Now that you know how to represent graphs in computer memory, the next obvious step is to learn the basic operations on a graph. The operations commonly performed on a graph are as follows:

1. Create the graph. That is, store the graph in computer memory using a particular graph representation.
2. Clear the graph. This operation makes the graph empty.
3. Determine whether the graph is empty.
4. Traverse the graph.
5. Print the graph.

We will add more operations on a graph when we discuss a specific application or a particular graph later in this chapter.

How a graph is represented in computer memory depends on the specific application. For illustration purposes, we use the adjacency list (linked list) representation of graphs. Therefore, for each vertex v the vertices adjacent to v (in a directed graph, also called the **immediate successors**) are stored in the linked list associated with v.

To manage the data in a linked list, we use the `class unorderedLinkedList`, discussed in Chapter 5.

The labeling of the vertices of a graph depends on a specific application. If you are dealing with the graph of cities, you could label the vertices by the names of the cities. However, to write algorithms to manipulate a graph as well as to simplify the algorithm, there must be some ordering to the vertices. That is, we must specify the first vertex, the second vertex, and so on. Therefore, for simplicity, throughout this chapter we assume that the n vertices of the graphs are numbered 0, 1, . . ., n - 1. Moreover, it follows that the class we will design to implement the graph algorithm will *not* be a template.

Graphs as ADTs

In this section, we describe the class to implement graphs as an abstract data type (ADT) and provide the definitions of the functions to implement the operations on a graph.

The following class defines a graph as an ADT:

```
//*************************************************************
// Author: D.S. Malik
//
// class graphType
// This class specifies the basic operations to implement a graph.
//*************************************************************

class graphType
{
public:
    bool isEmpty() const;
      //Function to determine whether the graph is empty.
      //Postcondition: Returns true if the graph is empty;
      //    otherwise, returns false.

    void createGraph();
      //Function to create a graph.
      //Postcondition: The graph is created using the
      //    adjacency list representation.

    void clearGraph();
      //Function to clear graph.
      //Postcondition: The memory occupied by each vertex
      //    is deallocated.

    void printGraph() const;
      //Function to print graph.
      //Postcondition: The graph is printed.

    void depthFirstTraversal();
      //Function to perform the depth first traversal of
      //the entire graph.
      //Postcondition: The vertices of the graph are printed
      //    using the depth first traversal algorithm.
```

```
    void dftAtVertex(int vertex);
      //Function to perform the depth first traversal of
      //the graph at a node specified by the parameter vertex.
      //Postcondition: Starting at vertex, the vertices are
      //    printed using the depth first traversal algorithm.

    void breadthFirstTraversal();
      //Function to perform the breadth first traversal of
      //the entire graph.
      //Postcondition: The vertices of the graph are printed
      //    using the breadth first traversal algorithm.

    graphType(int size = 0);
      //Constructor
      //Postcondition: gSize = 0; maxSize = size;
      //    graph is an array of pointers to linked lists.

    ~graphType();
      //Destructor
      //The storage occupied by the vertices is deallocated.

protected:
    int maxSize;    //maximum number of vertices
    int gSize;      //current number of vertices
    unorderedLinkedList<int> *graph; //array to create
                                     //adjacency lists

private:
    void dft(int v, bool visited[]);
      //Function to perform the depth first traversal of
      //the graph at a node specified by the parameter vertex.
      //This function is used by the public member functions
      //depthFirstTraversal and dftAtVertex.
      //Postcondition: Starting at vertex, the vertices are
      //    printed using the depth first traversal algorithm.
};
```

We leave the UML class diagram of the `class graphType` as an exercise.

The definitions of the functions of the `class graphType` are discussed next.

A graph is empty if the number of vertices is 0—that is, if `gSize` is `0`. Therefore, the definition of the function `isEmpty` is as follows:

```
bool graphType::isEmpty() const
{
    return (gSize == 0);
}
```

The definition of the function `createGraph` depends on how the data is input into the program. For illustration purposes, we assume that the data to the program is input from a file. The user is prompted for the input file. The data in the file appears in the following form:

```
5
0 2 4 ... -999
1 3 6 8 ... -999
...
```

The first line of input specifies the number of vertices in the graph. The first entry in the remaining lines specifies the vertex, and all of the remaining entries in the line (except the last) specify the vertices that are adjacent to the vertex. Each line ends with the number `-999`.

Using these conventions, the definition of the function `createGraph` is as follows:

```
void graphType::createGraph()
{
    ifstream infile;
    char fileName[50];

    int vertex;
    int adjacentVertex;

    if (gSize != 0)  //if the graph is not empty, make it empty
        clearGraph();

    cout << "Enter input file name: ";
    cin >> fileName;
    cout << endl;

    infile.open(fileName);

    if (!infile)
    {
        cout << "Cannot open input file." << endl;
        return;
    }

    infile >> gSize;   //get the number of vertices

    for (int index = 0; index < gSize; index++)
    {
        infile >> vertex;
        infile >> adjacentVertex;

        while (adjacentVertex != -999)
        {
            graph[vertex].insertLast(adjacentVertex);
            infile >> adjacentVertex;
        } //end while
    } // end for

    infile.close();
} //end createGraph
```

The function `clearGraph` empties the graph by deallocating the storage occupied by each linked list and then setting the number of vertices to 0.

```
void graphType::clearGraph()
{
    for (int index = 0; index < gSize; index++)
        graph[index].destroyList();

    gSize = 0;
} //end clearGraph
```

The definition of the function `printGraph` is given next:

```
void graphType::printGraph() const
{
    for (int index = 0; index < gSize; index++)
    {
        cout << index << " ";
        graph[index].print();
        cout << endl;
    }

    cout << endl;
} //end printGraph
```

The definitions of the constructor and the destructor are as follows:

```
    //Constructor
graphType::graphType(int size)
{
    maxSize = size;
    gSize = 0;
    graph = new unorderedLinkedList<int>[size];
}

    //Destructor
graphType::~graphType()
{
    clearGraph();
}
```

Graph Traversals

Processing a graph requires the ability to traverse the graph. This section discusses the graph traversal algorithms.

Traversing a graph is similar to traversing a binary tree, except that traversing a graph is a bit more complicated. Now a binary tree has no cycles and starting at the root node we can traverse the entire tree. On the other hand, a graph might have cycles and we might not be able to traverse the entire graph from a single vertex (for example, if the graph is not connected). Therefore, we must keep track of the vertices that have been visited. We must also traverse the graph from each vertex (that has not been visited) of the graph. This ensures that the entire graph is traversed.

The two most common graph traversal algorithms are the **depth-first traversal** and **breadth-first traversal**, which are described next. For simplicity, we assume that when a vertex is visited, its index is output. Moreover, each vertex is visited only once. We use the `bool` array `visited` to keep track of the visited vertices.

Depth-First Traversal

The **depth-first traversal** is similar to the preorder traversal of a binary tree. The general algorithm is as follows:

```
for each vertex, v, in the graph
   if v is not visited
      start the depth first traversal at v
```

Consider the graph G_3 of Figure 12-4. It is shown here again as Figure 12-7 for easy reference.

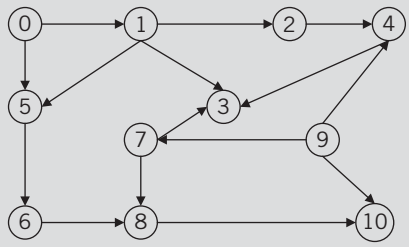


FIGURE 12-7 Directed graph G_3

The depth-first ordering of the vertices of graph G_3 in Figure 12-7 is as follows:

```
0 1 2 4 3 5 6 8 10 7 9
```

For the graph of Figure 12-7, the depth-first search starts at the vertex 0. After visiting all the vertices that can be reached starting at the vertex 0, the depth-first search starts at the next vertex that is not visited. There is a path from the vertex 0 to every other vertex except the vertices 7 and 9. Therefore, when the depth-first search starts at the vertex 0, all the vertices except 7 and 9 are visited before these vertices. After completing the depth-first search that started at the vertex 0, the depth-first search starts at the vertex 7 and then at the vertex 9. Note that there is no path from the vertex 7 to the vertex 9. Therefore, after completing the depth-first search that started at the vertex 7, the depth-first search starts at the vertex 9.
The general algorithm to do a depth-first traversal *at a given node*, *v*, is as follows:

1. `mark node v as visited`
2. `visit the node`
3.
```
for each vertex u adjacent to v
      if u is not visited
         start the depth first traversal at u
```

Clearly, this is a recursive algorithm. We use a recursive function, `dft`, to implement this algorithm. The vertex at which the depth-first traversal is to be started, and the `bool` array `visited`, are passed as parameters to this function.

```
void graphType::dft(int v, bool visited[])
{
    visited[v] = true;
    cout << " " << v << " ";  //visit the vertex

    linkedListIterator<int> graphIt;

        //for each vertex adjacent to v
    for (graphIt = graph[v].begin(); graphIt != graph[v].end();
                                     ++graphIt)
    {
        int w = *graphIt;
        if (!visited[w])
            dft(w, visited);
    } //end while
} //end dft
```

In the preceding code, note that the statement

```
linkedListIterator<int> graphIt;
```

declares `graphIt` to be an iterator. In the `for` loop, we use it to traverse a linked list (adjacency list) to which the pointer `graph[v]` points. Next, let us look at the statement

```
int w = *graphIt;
```

The expression `*graphIt` returns the label of the vertex, adjacent to the vertex `v`, to which `graphIt` points.

Next, we give the definition of the function `depthFirstTraversal` to implement the depth-first traversal of the graph.

```
void graphType::depthFirstTraversal()
{
    bool *visited; //pointer to create the array to keep
                   //track of the visited vertices
    visited = new bool[gSize];

    for (int index = 0; index < gSize; index++)
        visited[index] = false;

        //For each vertex that is not visited, do a depth
        //first traverssal
    for (int index = 0; index < gSize; index++)
        if (!visited[index])
            dft(index,visited);
    delete [] visited;
} //end depthFirstTraversal
```

The function `depthFirstTraversal` performs a depth-first traversal of the entire graph. The definition of the function `dftAtVertex`, which performs a depth-first traversal at a given vertex, is as follows:

```
void graphType::dftAtVertex(int vertex)
{
    bool *visited;

    visited = new bool[gSize];

    for (int index = 0; index < gSize; index++)
        visited[index] = false;

    dft(vertex, visited);

    delete [] visited;
} // end dftAtVertex
```

Breadth-First Traversal

The **breadth-first traversal** of a graph is similar to traversing a binary tree level-by-level (the nodes at each level are visited from left to right). All the nodes at any level, *i*, are visited before visiting the nodes at level *i* + 1.

The breadth-first ordering of the vertices of the graph G_3 in Figure 12-7 is as follows:

```
0 1 5 2 3 6 4 8 10 7 9
```

For the graph G_3, we start the breadth traversal at vertex `0`. After visiting the vertex `0`, we visit the vertices that are directly connected to it and are not visited, which are `1` and `5`. Next, we visit the vertices that are directly connected to `1` and are not visited, which are `2` and `3`. After this, we visit the vertices that are directly connected to `5` and are not visited, which is `6`. After this, we visit the vertices that are directly connected to `2` and are not visited, and so on.

As in the case of the depth-first traversal, because it might not be possible to traverse the entire graph from a single vertex, the breadth-first traversal also traverses the graph from each vertex that is not visited. Starting at the first vertex, the graph is traversed as much as possible; we then go to the next vertex that has not been visited. To implement the breadth-first search algorithm, we use a queue. The general algorithm is as follows:

1. `for` each vertex `v` in the graph
   `if v` is not visited
   add `v` to the `queue //start the breadth first search at v`
2. Mark `v` as `visited`
3. `while` the `queue` is not empty
   3.1. Remove vertex `u` from the `queue`
   3.2. Retrieve the vertices adjacent to `u`

3.3. for each vertex w that is adjacent to u
    if w is not visited
        3.3.1. Add w to the queue
        3.3.2. Mark w as visited

The following C++ function, breadthFirstTraversal, implements this algorithm:

```
void graphType::breadthFirstTraversal()
{
    linkedQueueType<int> queue;

    bool *visited;
    visited = new bool[gSize];

    for (int ind = 0; ind < gSize; ind++)
        visited[ind] = false;   //initialize the array
                                //visited to false

    linkedListIterator<int> graphIt;

    for (int index = 0; index < gSize; index++)
        if (!visited[index])
        {
            queue.addQueue(index);
            visited[index] = true;
            cout << " " << index << " ";

            while (!queue.isEmptyQueue())
            {
                int u = queue.front();
                queue.deleteQueue();

                for (graphIt = graph[u].begin();
                     graphIt != graph[u].end(); ++graphIt)
                {
                    int w = *graphIt;
                    if (!visited[w])
                    {
                        queue.addQueue(w);
                        visited[w] = true;
                        cout << " " << w << " ";
                    }
                }
            } //end while
        }

    delete [] visited;
} //end breadthFirstTraversal
```

As we continue to discuss graph algorithms, we will be writing C++ functions to implement specific algorithms, and so we will derive (using inheritance) new classes from the class graphType.

Shortest Path Algorithm

Graph theory has many applications. For example, we can use graphs to show how different chemicals are related or to show airline routes. Graphs also be used to show the highway structure of a city, state, or country. The edges connecting two vertices can be assigned a nonnegative real number, called the **weight of the edge**. If the graph represents a highway structure, the weight can represent the distance between two places or the travel time from one place to another. Such graphs are called **weighted graphs**.

Let G be a weighted graph. Let u and v be two vertices in G, and let P be a path in G from u to v. The **weight of the path** P is the sum of the weights of all the edges on the path P, which is also called the **weight** of v from u via P.

Let G be a weighted graph representing a highway structure. Suppose that the weight of an edge represents the travel time. For example, to plan monthly business trips, a salesperson wants to find the **shortest path** (that is, the path with the smallest weight) from her or his city to every other city in the graph. Many such problems exist in which we want to find the shortest path from a given vertex, called the **source**, to every other vertex in the graph.

This section describes the **shortest path algorithm**, also called a **greedy algorithm**, developed by Dijkstra.

Let G be a graph with n vertices, where $n \geq 0$. Let $V(G) = \{v_1, v_2, \ldots, v_n\}$. Let W be a two-dimensional $n \times n$ matrix such that

$$W(i,j) = \begin{cases} w_{ij} & \text{if } (v_i, v_j) \text{ is an edge in } G \text{ and } w_{ij} \text{ is the weight of the edge } (v_i, v_j) \\ \infty & \text{if there is no edge from } v_i \text{ to } v_j \end{cases}$$

The input to the program is the graph and the weight matrix associated with the graph. To make inputting the data easier, we extend the definition of the `class graphType` (using inheritance), and add the function `createWeightedGraph` to create the graph and the weight matrix associated with the graph. Let us call this `class weightedGraphType`. The functions to implement the shortest path algorithm will also be added to this class.

```
//*************************************************************
// Author: D.S. Malik
//
// class weightedGraphType
// This class specifies the operations to find the weight of the
// shortest path from a given vertex to every other vertex in a
// graph.
//*************************************************************

class weightedGraphType: public graphType
{
public:
    void createWeightedGraph();
      //Function to create the graph and the weight matrix.
      //Postcondition: The graph using adjacency lists and
      //    its weight matrix is created.
```

```
    void shortestPath(int vertex);
      //Function to determine the weight of a shortest path
      //from vertex, that is, source, to every other vertex
      //in the graph.
      //Postcondition: The weight of the shortest path from vertex
      //    to every other vertex in the graph is determined.

    void printShortestDistance(int vertex);
      //Function to print the shortest weight from the vertex
      //specified by the parameter vertex to every other vertex in
      //the graph.
      //Postcondition: The weight of the shortest path from vertex
      //    to every other vertex in the graph is printed.

    weightedGraphType(int size = 0);
      //Constructor
      //Postcondition: gSize = 0; maxSize = size;
      //    graph is an array of pointers to linked lists.
      //    weights is a two-dimensional array to store the weights
      //       of the edges.
      //    smallestWeight is an array to store the smallest weight
      //       from source to vertices.

    ~weightedGraphType();
      //Destructor
      //The storage occupied by the vertices and the arrays
      //weights and smallestWeight is deallocated.

protected:
    double **weights;   //pointer to create weight matrix
    double *smallestWeight; //pointer to create the array to store
                     //the smallest weight from source to vertices
};
```

We leave the UML class diagram of the `class weightedGraphType` and the inheritance hierarchy as an exercise. The definition of the function `createWeightedGraph` is also left as an exercise for you. Next, we describe the shortest path algorithm.

Shortest Path

Given a vertex, say `vertex` (that is, a source), this section describes the shortest path algorithm. The general algorithm is as follows:

1. Initialize the array `smallestWeight` so that

   ```
   smallestWeight[u] = weights[vertex, u]
   ```

2. Set `smallestWeight[vertex] = 0`.
3. Find the vertex `v` that is closest to `vertex` for which the shortest path has not been determined.
4. Mark `v` as the (next) vertex for which the smallest weight is found.

5. For each vertex `w` in `G`, such that the shortest path from `vertex` to `w` has not been determined and an edge `(v, w)` exists, if the weight of the path to `w` via `v` is smaller than its current weight, update the weight of `w` to the weight of `v` + the weight of the edge `(v, w)`.

Because there are *n* vertices, Steps 3 through 5 are repeated *n* - 1 times. Example 12-5 illustrates the shortest path algorithm. (We use the Boolean array `weightFound` to keep track of the vertices for which the smallest weight from the source vertex has been found. If the smallest weight for a vertex, from the source, has been found, then this vertex's corresponding entry in the array `weightFound` is set to `true`; otherwise the corresponding entry is `false`.)

EXAMPLE 12-5

Let *G* be the graph shown in Figure 12-8.

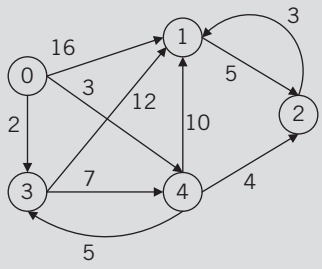
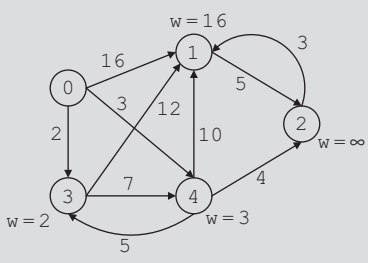


FIGURE 12-8 Weighted graph *G*

Suppose that the source vertex of *G* is `0`. The graph shows the weight of each edge. After Steps 1 and 2 execute, the resulting graph is as shown in Figure 12-9.



	[0]	[1]	[2]	[3]	[4]
smallestWeight	0	16	∞	2	3
	[0]	[1]	[2]	[3]	[4]
weightFound	T	F	F	F	F

FIGURE 12-9 Graph after Steps 1 and 2 execute

Iteration 1 of Steps 3 through 5: At Step 3, we select a vertex that is closest to the vertex `0` and for which the shortest path has not been found. We do this by finding a vertex in the array

`smallestWeight` that has the smallest weight and its corresponding entry in the array `weightFound` is `false`. Therefore, in this iteration, we select the vertex `3`. At Step 4, we mark `weightFound[3]` as `true`. Next at Step 5, we consider vertices `1` and `4` because these are the vertices for which there is an edge from the vertex `3` and the shortest path from `0` to these vertices has not been found. We then check if the path from the vertex `0` to the vertices `1` and `4` via the vertex `3` can be improved. The weight of the path `0-3-1` from `0` to `1` is less than the weight of the path `0-1`. So we update `smallestWeight[1]` to `14`. The weight of the path `0-3-4`, which is `2 + 7 = 9`, is greater than the weight of the path `0-4`, which is 3. So we do not update the weight of the vertex 4. Figure 12-10 shows the resulting graph. (The dotted arrow shows the shortest path from the source—that is, from `0`—to the vertex.)

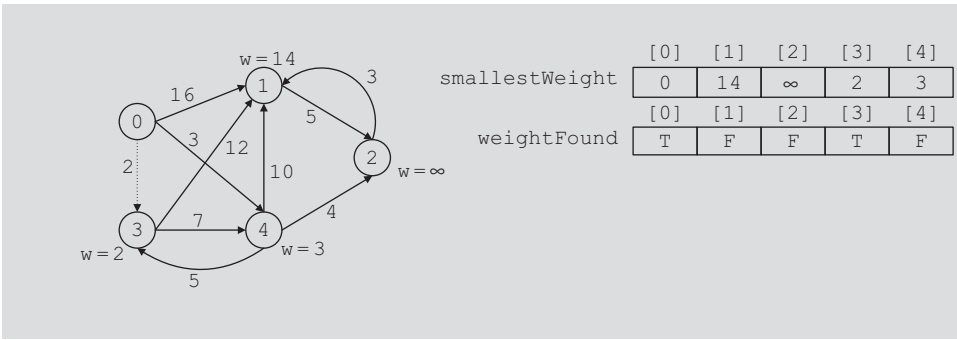


FIGURE 12-10 Graph after the first iteration of Steps 3 to 5

Iteration 2 of Steps 3 through 5: At Step 3, we select vertex `4` because this is the vertex in the array `smallestWeight` that has the smallest weight and its corresponding entry in the array `weightFound` is `false`. Next we execute Steps 4 and 5. At Step 4, we set `weightFound[4]` to `true`. At Step 5, we consider vertices `1` and `2` because these are the vertices for which there is an edge from the vertex `4` and the shortest path from `0` to these vertices has not been found. We then check if the path from the vertex `0` to the vertices `1` and `2` via the vertex `4` can be improved. Clearly, the weight of the path `0-4-1`, which is `13`, is smaller than the current weight of `1`, which is `14`. So we update `smallestWeight[1]`. Similarly, we update `smallestWeight[2]`. Figure 12-11 shows the resulting graph.

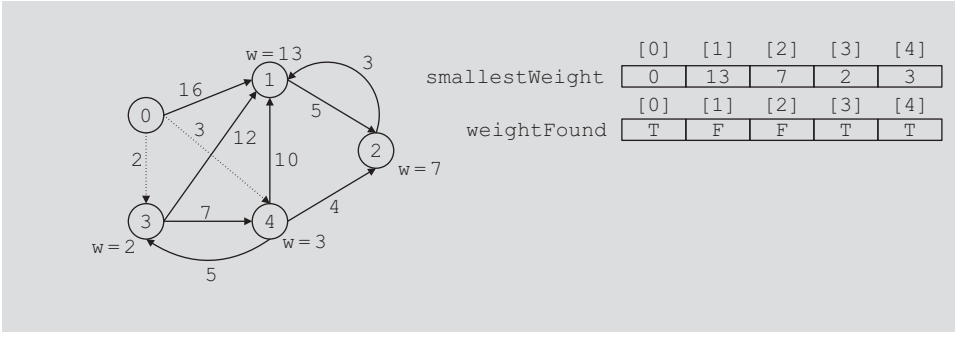


FIGURE 12-11 Graph after the second iteration of Steps 3 to 5

Iteration 3 of Steps 3 through 5: At Step 3, the vertex selected is `2`. At Step 4, we set `weightFound[2]` to `true`. Next at Step 5, we consider the vertex `1` because this is the vertex for which there is an edge from the vertex `2` and the shortest path from `0` to this vertex has not been found. We then check if the path from the vertex `0` to the vertex `1` via the vertex `2` can be improved. Clearly, the weight of the path `0-4-2-1`, which is `10`, from `0` to `1` is smaller than the current weight of `1` (which is `13`). So we update `smallestWeight[1]`. Figure 12-12 shows the resulting graph.

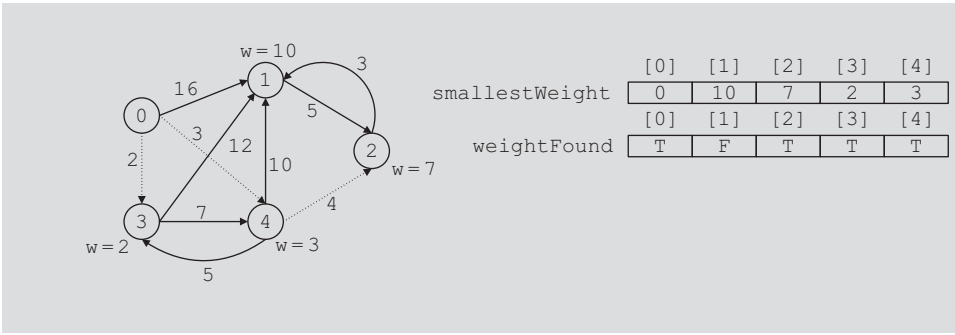


FIGURE 12-12 Graph after the third iteration of Steps 3 to 5

Iteration 4 of Steps 3 through 5: At Step 3, the vertex `1` is selected and at Step 4, `weightFound[1]` is set to `true`. In this iteration, the action of Step 5 is null because the shortest path from the vertex `0` to every other vertex in the graph has been determined. Figure 12-13 shows the final graph.

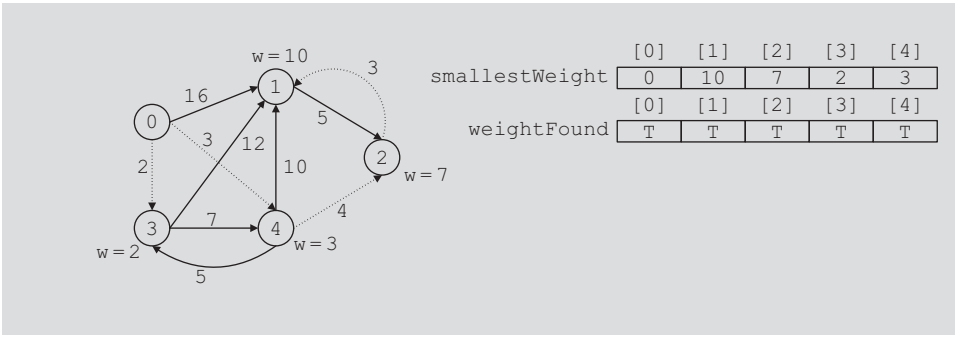


FIGURE 12-13 Graph after the fourth iteration of Steps 3 through 5

The following C++ function, `shortestPath`, implements the previous algorithm:

```
void weightedGraphType::shortestPath(int vertex)
{
    for (int j = 0; j < gSize; j++)
        smallestWeight[j] = weights[vertex][j];
```

```
    bool *weightFound;
    weightFound = new bool[gSize];

    for (int j = 0; j < gSize; j++)
        weightFound[j] = false;

    weightFound[vertex] = true;
    smallestWeight[vertex] = 0;

    for (int i = 0; i < gSize - 1; i++)
    {
        double minWeight = DBL_MAX;
        int v;

        for (int j = 0; j < gSize; j++)
            if (!weightFound[j])
                if (smallestWeight[j] < minWeight)
                {
                    v = j;
                    minWeight = smallestWeight[v];
                }

        weightFound[v] = true;

        for (int j = 0; j < gSize; j++)
            if (!weightFound[j])
                if (minWeight + weights[v][j] < smallestWeight[j])
                    smallestWeight[j] = minWeight + weights[v][j];
    } //end for
} //end shortestPath
```

Note that the function `shortestPath` records only the weight of the shortest path from the source to a vertex. We leave it for you to modify this function so that the shortest path from the source to a vertex is also recorded. Moreover, this function uses the named constant `DBL_MAX`, which is defined in the header file `cfloat`.

Let G be a graph with n vertices. In the function `shortestPath`, the first `for` loop executes n times and the second `for` loop also executes n times. The third `for` loop executes $n - 1$ times. The body of the third `for` loop contains two `for` loops, in sequence, and each of these `for` loops executes n times. Thus, the total number of iterations of the `for` loops is $n + n + (n - 1)(n + n) = 2n + 2n(n - 1) = O(n^2)$. Hence, the function `shortestPath`, that is, the shortest path algorithm is of order $O(n^2)$.

The definitions of the function `printShortestDistance` and the constructor and destructor are as follows:

```
void weightedGraphType::printShortestDistance(int vertex)
{
    cout << "Source Vertex: " << vertex << endl;
    cout << "Shortest distance from source to each vertex."
         << endl;
    cout << "Vertex  Shortest_Distance" << endl;
```

```
    for (int j = 0; j < gSize; j++)
        cout << setw(4) << j << setw(12) << smallestWeight[j]
             << endl;
    cout << endl;
} //end printShortestDistance

    //Constructor
weightedGraphType::weightedGraphType(int size)
                  :graphType(size)
{
    weights = new double*[size];

    for (int i = 0; i < size; i++)
        weights[i] = new double[size];

    smallestWeight = new double[size];
}

    //Destructor
weightedGraphType::~weightedGraphType()
{
    for (int i = 0; i < gSize; i++)
       delete [] weights[i];

    delete [] weights;
    delete smallestWeight;
}
```

Minimum Spanning Tree

Consider the graph of Figure 12-14, which represents the airline connections of a company, between seven cities. The number on each edge represents some cost factor of maintaining the connection between cities.

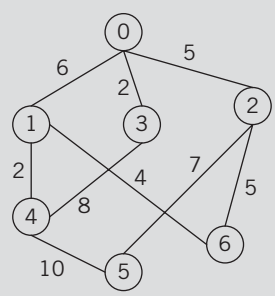


FIGURE 12-14 Airline connections between cities and the cost factor of maintaining the connections

Due to financial hardship, the company needs to shut down the maximum number of connections and still be able to fly (may be not directly) from one city to another. The graphs of Figure 12-15(a), (b), and (c) show three different solutions.

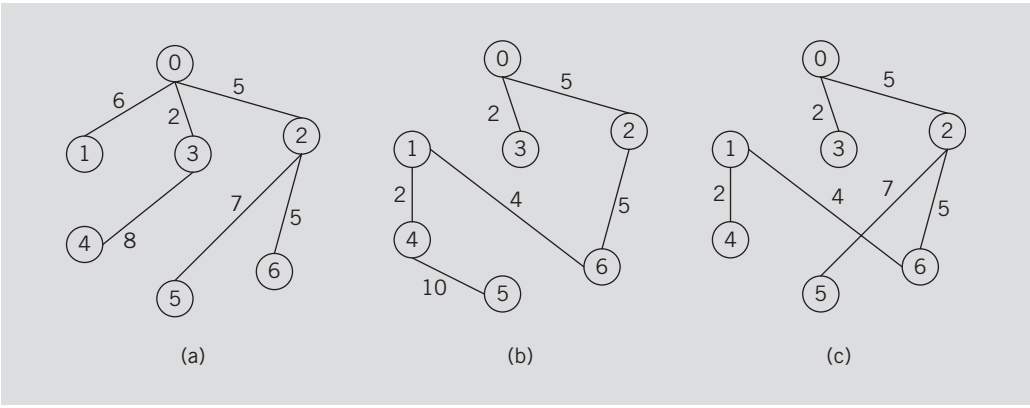


FIGURE 12-15 Possible solutions to the graph of Figure 12-14

The total cost factor of maintaining the remaining connections in Figure 12-15(a) is 33, in Figure 12-15(b) it is 28, and in Figure 12-15(c) it is 25. Out of these three solutions, obviously, the desired solution is the one shown by the graph of Figure 12-15(c) because it gives the lowest cost factor. Graphs of Figure 12-15 are called spanning trees of the graph of Figure 12-14.

Let us note the following from the graphs of Figure 12-15. Each of the graphs of Figure 12-15 is a subgraph of the graph of Figure 12-14, and there is a unique path from a node to any other node. Such graphs are called trees. There are many other situations, where given a weighted graph, we need to determine a graph such as in Figure 12-15 with the smallest weight. In this section, we give an algorithm to determine such graphs. However, first we introduce some terminology.

A (**free**) **tree** T is a simple graph such that if u and v are two vertices in T, there is a unique path from u to v. A tree in which a particular vertex is designated as a root is called a **rooted tree**. If a weight is assigned to the edges in T, T is called a **weighted tree**. If T is a weighted tree, the **weight** of T, denoted by $W(T)$, is the sum of the weights of all the edges in T.

A tree T is called a **spanning tree** of graph G if T is a subgraph of G such that $V(T) = V(G)$, that is, all the vertices of G are in T.

Suppose G denotes the graph of Figure 12-14. Then the graphs of Figure 12-15 show three spanning trees of G. Let us note the following theorem.

Theorem 12-1: A graph G has a spanning tree if and only if G is connected.

From this theorem, it follows that to determine a spanning tree of a graph, the graph must be connected.

Definition: Let G be a weighted graph. A **minimum (minimal) spanning tree** of G is a spanning tree with the minimum weight.

There are two well-known algorithms, Prim's algorithm and Kruskal's algorithm, for finding a minimum spanning tree of a graph. This section discusses Prim's algorithm to find a minimum spanning tree. If interested, you can find Kruskal's algorithm in the discrete structures book or a data structures book listed in Appendix H.

Prim's algorithm builds the tree iteratively by adding edges until a minimum spanning tree is obtained. We start with a designated vertex, which we call the source vertex. At each iteration, a new edge that does not form a cycle is added to the tree.

Let G be a weighted graph such that $V(G) = \{v_0, v_1, \ldots, v_{n-1}\}$, where n, the number of vertices, is nonnegative. Let v_0 be the source vertex. Let T be the partially built tree. Initially $V(T)$ contains the source vertex and $E(T)$ is empty. At the next iteration, a new vertex that is not in $V(T)$ is added to $V(T)$, such that an edge exists from a vertex in T to the new vertex so that the corresponding edge has the smallest weight. The corresponding edge is added to $E(T)$.

The general form of Prim's algorithm is as follows. (Let n be the number of vertices in G.)

1. Set V(T) = {source}
2. Set E(T) = empty
3. `for i = 1 to n`

   3.1. `minWeight = infinity;`

   3.2.
```
for j = 1 to n
     if vj is in V(T)
       for k = 1 to n
         if vk is not in T and weight[vj, vk] < minWeight
         {
            endVertex = vk;
            edge = (vj, vk);
            minWeight = weight[vj, vk];
         }
```

   3.3. `V(T) = V(T) ∪ {endVertex};`

   3.4. `E(T) = E(T) ∪ {edge};`

Let us illustrate Prim's algorithm using the graph G of Figure 12-16 (which is same as the graph of Figure 12-14).

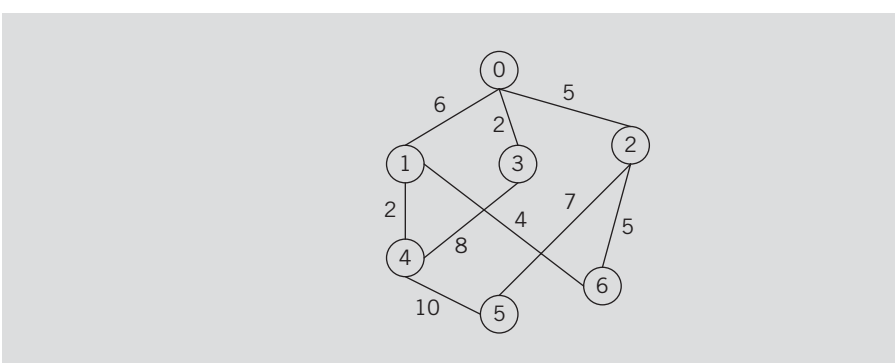


FIGURE 12-16 Weighted graph *G*

Let N denote the set of vertices of G that are not in T. Suppose that the source vertex is `0`. Figure 12-17 shows how Prim's algorithm works.

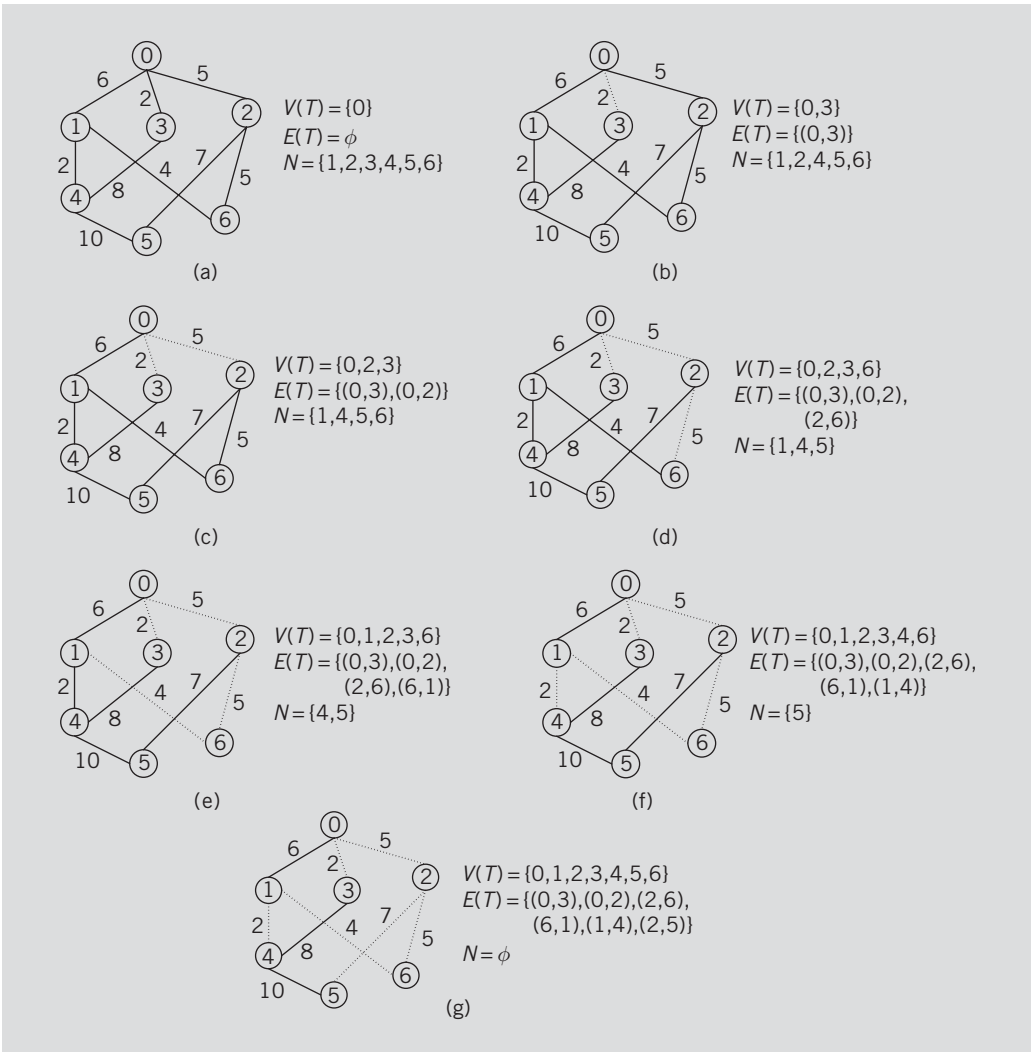


FIGURE 12-17 Graph G, $V(T)$, $E(T)$, and N after Steps 1 and 2 execute

After Steps 1 and 2 execute, $V(T)$, $E(T)$, and N are as shown in Figure 12-17(a). Step 3 checks the following edges: (0,1), (0,2), and (0,3). The weight of the edge (0,1) is 6, the weight of the edge (0,2) is 5, and the weight of the edge (0,3) is 2. Clearly, the edge (0,3) has the smallest weight; see Figure 12-17(b). Therefore, vertex 3 is added to $V(T)$ and the edge (0,3) is added to $E(T)$. Figure 12-17(b) shows the resulting graph, $V(T)$, $E(T)$, and N. (The dotted line shows the edge in T.)

Next, Step 3 checks the following edges: (0,1), (0,2), and (3,4). The weight of the edge (0,1) is 6, the weight of the edge (0,2) is 5, and the weight of the edge (3,4) is 8. Clearly, the edge (0,2) has the smallest weight. Therefore, vertex `2` is added to $V(T)$ and the edge (0,2) is added to $E(T)$. Figure 12-17(c) shows the resulting graph, $V(T)$, $E(T)$, and N.

At the next iteration, Step 3 checks the following edges: (0,1), (2,5), (2,6), and (3,4). The weight of the edge (0,1) is 6, the weight of the edge (2,5) is 7, the weight of the edge (2,6) is 5, and the weight of the edge (3,4) is 8. Clearly, the edge (2,6) has the smallest weight. Therefore, vertex `6` is added to $V(T)$ and the edge (2,6) is added to $E(T)$. Figure 12-17(d) shows the resulting graph, $V(T)$, $E(T)$, and N. (The dotted lines show the edges in T.)

At the next iteration, Step 3 checks the following edges: (0,1), (2,5), (3,4), and (6,1). The weight of the edge (0,1) is 6, the weight of the edge (2,5) is 7, the weight of the edge (3,4) is 8, and the weight of the edge (6,1) is 4. Clearly, the edge (6,1) has the smallest weight. Therefore, vertex `1` is added to $V(T)$ and the edge (6,1) is added to $E(T)$. Figure 12-17(e) shows the resulting graph, $V(T)$, $E(T)$, and N. (The dotted lines show the edges in T.)

At the next iteration, Step 3 checks the following edges: (1,4), (2,5), and (3,4). The weight of the edge (1,4) is 2, the weight of the edge (2,5) is 7, and the weight of the edge (3,4) is 8. Clearly, the edge (1,4) has the smallest weight. Therefore, vertex `4` is added to $V(T)$ and the edge (1,4) is added to $E(T)$. Figure 12-17(f) shows the resulting graph, $V(T)$, $E(T)$, and N. (The dotted lines show the edges in T.)

At the next iteration, Step 3 checks the following edges: (2,5) and (4,5). The weight of the edge (2,5) is 7 and the weight of the edge (4,5) is 10. Clearly, the edge (2,5) has the smallest weight. Therefore, vertex 5 is added to $V(T)$ and the edge (2,5) is added to $E(T)$. Figure 12-17(g) shows the resulting graph, $V(T)$, $E(T)$, and N. (The dotted lines show the edges in T.)

In Figure 12-17(g), the dotted lines show a minimum spanning tree of G of weight 25.

Before we give the definition of the function to implement Prim's algorithm, let us first define a spanning tree as an ADT.

Let `mstv` be a `bool` array such that `mstv[j]` is `true` if the vertex v_i is in T, and `false` otherwise. Let `edges` be an array such that `edges[j] = k`, if there is an edge connecting vertices v_j and v_k. Suppose that the edge (v_i, v_j) is in the minimum spanning tree. Let `edgeWeights` be an array such that `edgeWeights[j]` is the weight of the edge (v_i, v_j).

Using these conventions, the following class defines a spanning tree as an ADT:

```
//*****************************************************************
// Author: D.S. Malik
//
// class msTreeType
// This class specifies the operations to find a minimum
// spanning tree in a graph.
//*****************************************************************
```

```
class msTreeType: public graphType
{
public:
    void createSpanningGraph();
      //Function to create the graph and the weight matrix.
      //Postcondition: The graph using adjacency lists and
      //    its weight matrix is created.

    void minimumSpanning(int sVertex);
      //Function to create a minimum spanning tree with
      //root as sVertex.
      // Postcondition: A minimum spanning tree is created.
      //     The weight of the edges is also saved in the array
      //     edgeWeights.

    void printTreeAndWeight();
      //Function to output the edges of the minimum spanning tree
      //and the weight of the minimum spanning tree.
      //Postcondition: The edges of a minimum spanning tree
      //    and their weights are printed.

    msTreeType(int size = 0);
      //Constructor
      //Postcondition: gSize = 0; maxSize = size;
      //    graph is an array of pointers to linked lists.
      //    weights is a two-dimensional array to store the weights
      //      of the edges.
      //    edges is an array to store the edges of a minimum
      //      spanning tree.
      //    edgeWeights is an array to store the weights of the
      //      edges of a minimum spanning tree.

    ~msTreeType();
      //Destructor
      //The storage occupied by the vertices and the arrays
      //weights, edges, and edgeWeights is deallocated.

protected:
    int source;
    double **weights;
    int *edges;
    double *edgeWeights;
};
```

We leave the UML class diagram of the `class msTreeType` and the inheritance hierarchy as an exercise. The definition of the function `createSpanningGraph` is also left as an exercise for you. This function creates the graph and the weight matrix associated with the graph.

The following C++ function, `minimumSpanning`, implements Prim's algorithm, as described previously:

```
void msTreeType::minimumSpanning(int sVertex)
{
    int startVertex, endVertex;
    double minWeight;

    source = sVertex;

    bool *mstv;
    mstv = new bool[gSize];

    for (int j = 0; j < gSize; j++)
    {
        mstv[j] = false;
        edges[j] = source;
        edgeWeights[j] = weights[source][j];
    }

    mstv[source] = true;
    edgeWeights[source] = 0;

    for (int i = 0; i < gSize - 1; i++)
    {
        minWeight = DBL_MAX;

        for (int j = 0; j < gSize; j++)
            if (mstv[j])
                for (int k = 0; k < gSize; k++)
                    if (!mstv[k] && weights[j][k] < minWeight)
                    {
                        endVertex = k;
                        startVertex = j;
                        minWeight = weights[j][k];
                    }

        mstv[endVertex] = true;
        edges[endVertex] = startVertex;
        edgeWeights[endVertex] = minWeight;
    } //end for
} //end minimumSpanning
```

The definition of the function `minimumSpanning` contains three nested `for` loops. Therefore, in the worst case, Prim's algorithm given in this section is of the order $O(n^3)$. It is possible to design Prim's algorithm so that it is of the order $O(n^2)$; Programming Exercise 5 at the end of this chapter asks you to do this.

The definition of the function `printTreeAndWeight` is as follows:

```
void msTreeType::printTreeAndWeight()
{
    double treeWeight = 0;

    cout << "Source Vertex: " << source << endl;
    cout << "Edges    Weight" << endl;
```

```
    for (int j = 0; j < gSize; j++)
    {
        if (edges[j] != j)
        {
            treeWeight = treeWeight + edgeWeights[j];
            cout << "("<<edges[j] << ", " << j << ")    "
                 << edgeWeights[j] << endl;
        }
    }

    cout << endl;
    cout << "Minimum spanning Tree Weight: "
         << treeWeight << endl;
} //end printTreeAndWeight
```

The definitions of the constructor and the destructor are as follows:

```
msTreeType::msTreeType(int size)
           :graphType(size)
{
    weights = new double*[size];

    for (int i = 0; i < size; i++)
        weights[i] = new double[size];

    edges  = new int[size];

    edgeWeights = new double[size];
}

    //Destructor
msTreeType::~msTreeType()
{
    for (int i = 0; i < gSize; i++)
        delete [] weights[i];

    delete [] weights;
    delete [] edges;
    delete edgeWeights;
}
```


Topological Order

In college, before taking a particular course, students, usually, must take all its prerequisite courses, if any. For example, before taking the Programming II course, the student must take the Programming I course. However, certain courses can be taken independent of each other. The courses within a department can be represented as a directed graph. A directed edge from, say vertex *u* to vertex *v* means the course represented by the vertex *u* is a prerequisite of the course represented by the vertex *v*. It would be helpful for the students to know, before starting a major, the sequence in which they can take the courses so that before taking a course they take all its prerequisite courses and fulfill the

graduation requirements on time. In this section, we describe an algorithm that can be used to output the vertices of a directed graph in such a sequence. Let us first introduce some terminology.

Let G be a directed graph and $V(G) = \{v_1, v_2, \ldots, v_n\}$, where $n \geq 0$. A **topological ordering** of $V(G)$ is a linear ordering $v_{i1}, v_{i2}, \ldots, v_{in}$ of the vertices such that, if v_{ij} is a predecessor of v_{ik}, $j \neq k$, $1 \leq j \leq n$, $1 \leq k \leq n$, then v_{ij} precedes v_{ik}, that is, $j < k$ in this linear ordering.

In this section, we describe an algorithm, topological order, which outputs the vertices of a directed graph in topological order. We assume that the graph has no cycles. We leave it as an exercise for you to modify the algorithm for the graphs that have cycles.

Because the graph has no cycles, the following is true:

- There exists a vertex v in G such that v has no successor.
- There exists a vertex u in G such that u has no predecessor.

Suppose that the array `topologicalOrder` (of size n, the number of vertices) is used to store the vertices of G in topological order. Thus, if a vertex, say u, is a successor of the vertex v and `topologicalOrder[j]` = v and `topologicalOrder[k]` = u, then j < k.

The topological sort algorithm can be implemented either using the depth-first traversal or the breadth-first traversal. This section discusses how to implement topological ordering using the breadth-first traversal. Programming Exercise 7 at the end of this chapter describes how to implement the topological sort using the depth-first traversal.

We extend the definition of the `class graphType` (using inheritance) to implement the breadth-first topological ordering algorithm. Let us call this `class topologicalOrderType`. Next, we give the definition of the class that includes the functions to implement the topological ordering algorithm.

```
//*****************************************************************
// Author: D.S. Malik
//
// class topologicalOrderType
// This class specifies the operations to find a topological
// ordering of a graph.
//*****************************************************************

class topologicalOrderType: public graphType
{
public:
    void bfTopOrder();
      //Function to perform breadth first topological ordering of
      //a graph.
      //Postcondition: The vertices are output in a breadth first
      //topological order.
```

```
    topologicalOrderType(int size = 0);
      //Constructor
      //Postcondition: gSize = 0; maxSize = size;
      //    graph is an array of pointers to linked lists.
};
```

Next, we discuss how to implement the function `bfTopOrder`.

Breadth-First Topological Ordering

Recall that the breadth-first traversal algorithm is similar to traversing a binary tree level-by-level, and so the root node (which has no predecessor) is visited first. Therefore, in the breadth-first topological ordering, we first find a vertex that has no predecessor vertex and place it first in the topological ordering. We next find the vertex, say `v`, all of whose predecessors have been placed in the topological ordering and place `v` next in the topological ordering. To keep track of the number of vertices of a vertex, we use the array `predCount`. Initially, `predCount[j]` is the number of predecessors of the vertex v_j. The queue used to guide the breadth-first traversal is initialized to those vertices v_k such that `predCount[k]` is 0. In essence, the general algorithm is as follows:

1. Create the array `predCount` and initialize it so that `predCount[i]` is the number of predecessors of the vertex v_i.
2. Initialize the queue, say `queue`, to all those vertices v_k so that `predCount[k]` is 0. (Clearly, `queue` is not empty because the graph has no cycles.)
3. `while` the queue is not empty
   - 3.1. Remove the front element, `u`, of the queue.
   - 3.2. Put `u` in the next available position, say `topologicalOrder[topIndex]`, and increment `topIndex`.
   - 3.3. For all the immediate successors `w` of `u`,
     - 3.3.1. Decrement the predecessor count of `w` by `1`.
     - 3.3.2. `if` the predecessor count of `w` is 0, add `w` to `queue`.

The graph G_3 of Figure 12-7 has no cycles. The vertices of G_3 in breadth-first topological ordering are as follows:

```
Breadth First Topological order: 0 9 1 7 2 5 4 6 3 8 10
```

Next, we illustrate the breadth-first topological ordering of the graph G_3.

After Steps 1 and 2 execute, the arrays `predCount`, `topologicalOrder`, and `queue` are as shown in Figure 12-18. (Notice that for simplicity, we show only the elements of the queue.)

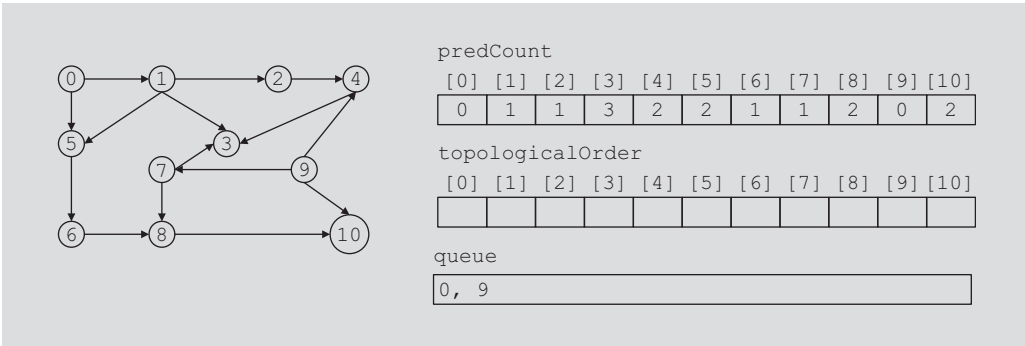


FIGURE 12-18 Arrays `predCount`, `topologicalOrder`, and `queue` after Steps 1 and 2 execute

Step 3 executes as long as the queue is nonempty.

Iteration 1 of Step 3: After Step 3.1 executes, the value of `u` is `0`. Step 3.2 stores the value of `u`, which is `0`, in the next available position in the array `topologicalOrder`. Notice that `0` is stored at position `0` in this array. Step 3.3 reduces the predecessor count of all the successors of `0` by `1`, and if the predecessor count of any successor node of `0` reduces to `0`, that node is pushed into `queue`. The successor nodes of the node `0` are the nodes `1` and `5`. The predecessor count of the node `1` reduces to `0`, and the predecessor count of the node `5` reduces to `1`. The node `1` is pushed into `queue`. After the first iteration of Step 3, the arrays `predCount`, `topologicalOrder`, and `queue` are as shown in Figure 12-19.

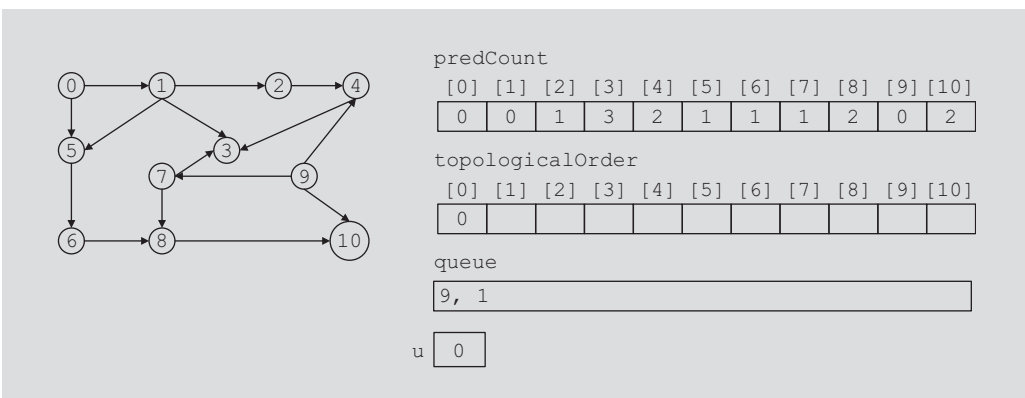


FIGURE 12-19 Arrays `predCount`, `topologicalOrder`, and `queue` after the first iteration of Step 3

Iteration 2 of Step 3: The queue is nonempty. After Step 3.1 executes, the value of `u` is `9`. Step 3.2 stores the value of `u`, which is `9`, in the next available position in the array `topologicalOrder`. Notice that `9` is stored at position `1` in this array. Step 3.3 reduces the

predecessor count of all the successors of 9 by 1, and if the predecessor count of any successor node of 9 reduces to 0, that node is pushed into queue. The successor nodes of the node 9 are the nodes 4, 7, and 10. The predecessor count of the node 7 reduces to 0 and the predecessor count of the nodes 4 and 10 reduces to 1. The node 7 is pushed into queue. After the second iteration of Step 3, the arrays predCount, topologicalOrder, and queue are as shown in Figure 12-20.

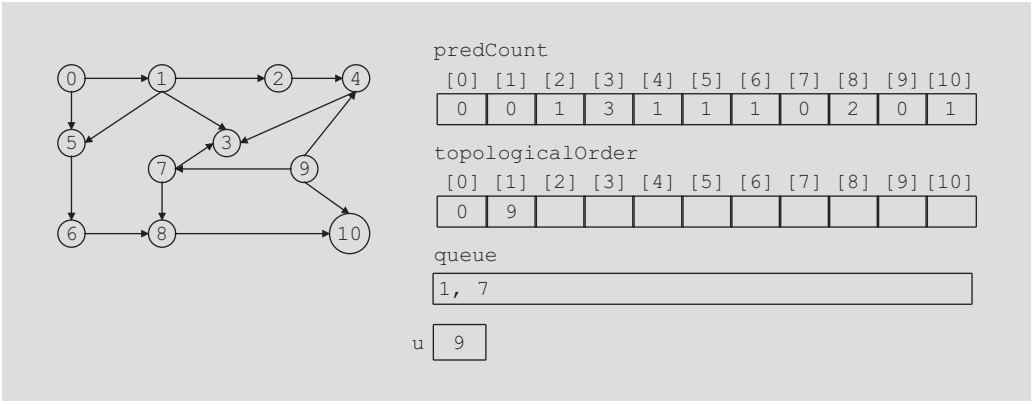


FIGURE 12-20 Arrays predCount, topologicalOrder, and queue after the second iteration of Step 3

Iteration 3 of Step 3: The queue is nonempty. After Step 3.1 executes, the value of u is 1. Step 3.2 stores the value of u, which is 1, in the next available position in the array topologicalOrder. Notice that 1 is stored at position 2 in this array. Step 3.3 reduces the predecessor count of all the successors of 1 by 1 and if the predecessor count of any successor node of 1 reduces to 0, that node is pushed into queue. The successor nodes of the node 1 are the nodes 2, 3, and 5. The predecessor count of the nodes 2 and 5 reduces to 0 and the predecessor count of the node 3 reduces to 2. The nodes 2 and 5, in this order, are pushed into the queue. After the third iteration of Step 3, the arrays predCount, topologicalOrder, and queue are as shown in Figure 12-21.

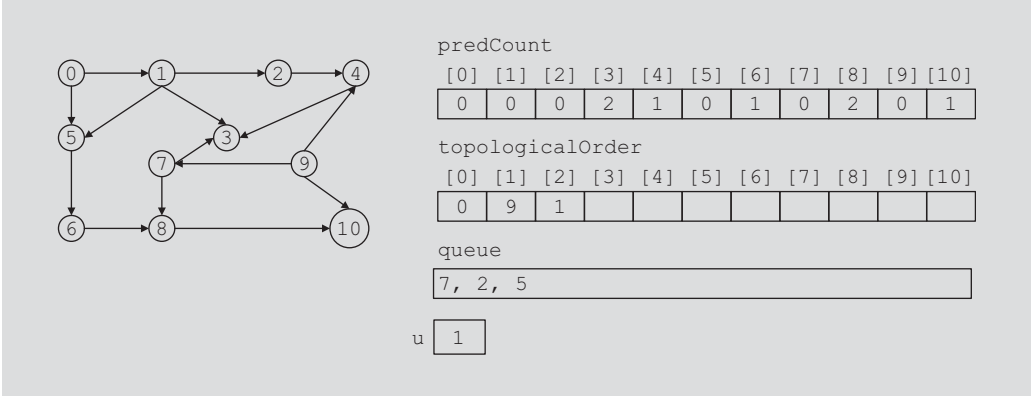


FIGURE 12-21 Arrays predCount, topologicalOrder, and queue after the third iteration of Step 3

If you repeat Step 3 eight more times, the arrays **predCount**, **topologicalOrder**, and **queue** are as shown in Figure 12-22.

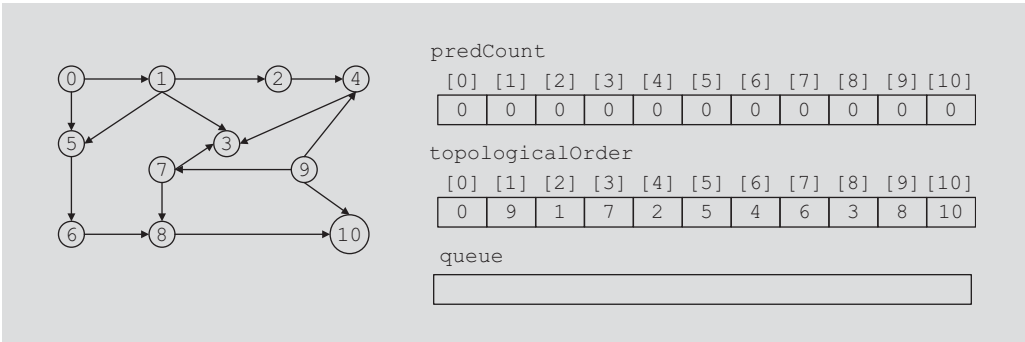


FIGURE 12-22 Arrays `predCount`, `topologicalOrder`, and `queue` after Step 3 executes

In Figure 12-22, the array **topologicalOrder** shows the breadth-first topological ordering of the nodes of the graph G_3.

The following C++ function implements this breadth-first topological ordering algorithm:

```
void topologicalOrderType::bfTopOrder()
{
    linkedQueueType<int> queue;

    int *topologicalOrder; //pointer to the array to store
                           //breadth first topological ordering
    topologicalOrder = new int[gSize];

    int topIndex = 0;

    linkedListIterator<int> graphIt; //iterator to traverse a
                                     //linked list

    int *predCount;    //pointer to the array to store the
                       //predecessor count of a vertex.
    predCount = new int[gSize];

    for (int ind = 0; ind < gSize; ind++)
        predCount[ind] = 0;

        //Determine the predecessor count of all the vertices.
    for (int ind = 0; ind < gSize; ind++)
    {
        for (graphIt = graph[ind].begin();
             graphIt != graph[ind].end(); ++graphIt)
        {
            int w = *graphIt;
            predCount[w]++;
        }
    }
```

```
        //Initialize queue: If the predecessor count of
        //vertex is 0, put this node into the queue.
    for (int ind = 0; ind < gSize; ind++)
        if (predCount[ind] == 0)
           queue.addQueue(ind);

    while (!queue.isEmptyQueue())
    {
        int u = queue.front();
        queue.deleteQueue();
        topologicalOrder[topIndex++] = u;

           //Reduce the predecessor count of all the successors
           //of u by 1. If the predecessor count of a vertex
           //becomes 0, put this vertex into the queue.
        for (graphIt = graph[u].begin();
             graphIt != graph[u].end(); ++graphIt)
        {
            int w = *graphIt;
            predCount[w]--;
            if (predCount[w] == 0)
               queue.addQueue(w);
        }
    }//end while

       //output the vertices in breadth first topological order
    for (int ind = 0; ind < gSize; ind++)
        cout << topologicalOrder[ind] << " ";

    cout << endl;

    delete [] topologicalOrder;
    delete [] predCount;
}//bfTopOrder
```

We leave the definition of the constructor as an exercise. (See Programming Exercise 6 at the end of this chapter.)

Euler Circuits

Let us consider the Königsberg bridge problem stated at the beginning of the chapter. The problem is to determine whether it is possible to take a walk that crosses each bridge exactly once before returning to the starting point; see Figure 12-1. As remarked earlier, Euler converted this problem into a graph theory problem as follows: Each of the islands *A*, *B*, *C*, and *D* is considered as a vertex of a graph and the bridges are considered as edges, as shown in Figure 12-2. Now the problem reduces to finding a circuit in the graph of Figure 12-2, such that it contains all the edges. In this section, we further describe properties of graphs, which will help us answer this question.

Definition: A **circuit** is a path of nonzero length from a vertex u to u with no repeated edges.

Definition: A circuit in a graph that includes all the edges of the graph is called an **Euler circuit**.

Definition: A graph G is said to be **Eulerian** if either G is a trivial graph or G has an Euler circuit.

Notice that the graph of Figure 12-2 is a connected graph and this graph has odd degree vertices as well as even degree vertices.

Let us consider a connected graph with more than one vertex such that every vertex has odd degree. For example, consider the graph of Figure 12-23. This is a connected graph and every vertex of this graph is odd degree. This graph has no circuit and so has no circuit that contains all the edges.

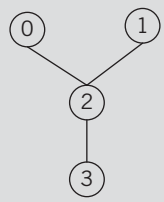


FIGURE 12-23 A graph with all vertices of odd degree

Next consider the connected graph G of Figure 12-24 such that every vertex has even degree.

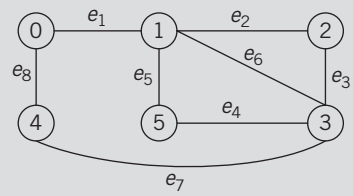


FIGURE 12-24 A graph with all vertices of even degree

The graph of Figure 12-24 has an Euler circuit. For example, (0, e_1, 1, e_2, 2, e_3, 3, e_4, 5, e_5, 1, e_6, 3, e_7, 4, e_8, 0) is an Euler circuit in the graph of Figure 12-24.

The following theorems give necessary and sufficient conditions for a connected graph to have an Euler circuit.

Theorem 12-2: If a connected graph G is Eulerian, then every vertex of G has even degree.

Theorem 12-3: Let G be a connected graph such that every vertex of G is of even degree. Then G has an Euler circuit.

We can effectively use this theorem to determine whether a connected graph G has an Euler circuit by checking whether all of its vertices are of even degree.

Let us again consider the Königsberg bridge problem. Notice that the graph corresponding to this problem is a connected graph but has vertices of odd degree; see Figure 12-2. Hence, by Theorem 12-2, the graph of Figure 12-2 has no Euler circuit. In other words, starting at one land area, it is not possible to walk across all the bridges exactly once and return to the starting land area.

After 1736, two additional bridges have been constructed on the Pregel river—one is between the regions B and C and the other is between A and D. The graph with the additional two bridges is shown in Figure 12-25.

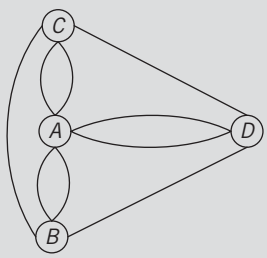


FIGURE 12-25 Graph of the Königsberg bridge problem with two additional bridges

By Theorem 12-3, every connected graph with only even degree vertices has an Euler circuit. Next, we describe an algorithm, known as Fleury's algorithm, which can be used to construct an Euler circuit in a connected graph with vertices of even degrees.

Fleury's Algorithm

Step 1. Choose a vertex v as the starting vertex for the circuit and choose an edge e with v as one of the end vertices.

Step 2. If the other end vertex u of the edge e is also v, go to Step 3. Otherwise, choose an edge e_1 different from e with u as one of the end vertices. If the other vertex u_1 of e_1 is v, go to Step 3; otherwise, choose an edge e_2 different from e and e_1 with u_1 as one of the end vertices and repeat Step 2.

Step 3. If the circuit T_1 obtained in Step 2 contains all the edges, then stop. Otherwise, choose an edge e_j different from the edges of T_1 such that one of the end vertices of e_j, say, w is a member of the circuit T_1.

Step 4. Construct a circuit T_2 with starting vertex w, as in Steps 1 and 2, such that all the edges of T_2 are different from the edges in the circuit T_1.

Step 5. Construct the circuit T_3 by inserting the circuit T_2 at w of the circuit T_1. Now go to Step 3 and repeat Step 3 with the circuit T_3.

Next, we illustrate how Fleury's algorithm works. Consider the graph of Figure 12-26.

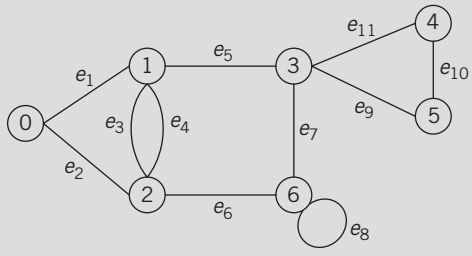


FIGURE 12-26 A graph with all vertices of even degree

Let us apply Fleury's algorithm to find an Eulerian circuit.

First, select vertex 0 and form the circuit: T_1: (0, e_1, 1, e_3, 2, e_2, 0).

Next, select vertex 1 and edge e_4. Construct the circuit: C_1: (1, e_4, 2, e_6, 6, e_7, 3, e_5, 1).

Then form the circuit: T_2: (0, e_1, C_1, e_3, 2, e_2, 0).

Circuit T_2 does not contain all the edges of the given graph. Now choose vertex 6 and edge e_8 and form the circuit: C_2: (6, e_8, 6).

Now construct the circuit: $T_{3:}$ (0, e_1, 1, e_4, 2, e_6, $C_{2,}$ e_7, 3, e_5, 1, e_3, 2, e_2, 0).

This circuit also does not contain all the edges. Select now vertex 3 and edge e_{11}. Form the circuit: C_3: (3, e_{11}, 4, e_{10}, 5, e_9, 3).

Next, construct the circuit: T_4: (0, e_1,1, e_4, 2, e_6, C_2, e_7, C_3, e_5, 1, e_3, 2, e_2, 0).

Circuit T_4 contains all the vertices and all the edges of the given graph and, hence, it is an Euler circuit.

We leave it as an exercise for you to write a program to implement Fleury's algorithm. (See Programming Exercise 8 at the end of this chapter.)

QUICK REVIEW

1. A graph G is a pair, $G = (V, E)$, where V is a finite nonempty set, called the set of vertices of G and $E \subseteq V \times V$, called the set of edges.
2. In an *undirected* graph $G = (V, E)$, the elements of E are unordered pairs.

3. In a directed graph $G = (V, E)$, the elements of E are ordered pairs.
4. Let G be a graph. A graph H is called a subgraph of G if every vertex of H is a vertex of G and every edge in H is an edge in G.
5. Two vertices u and v in an undirected graph are called adjacent if there is an edge from one to the other.
6. An edge incident on a single vertex is called a loop.
7. In an undirected graph, if two edges e_1 and e_2 are associated with the same pair of vertices $\{u, v\}$, then e_1 and e_2 are called parallel edges.
8. A graph is called a simple graph if it has no loops and no parallel edges.
9. Let $e = (u, v)$ be an edge in an undirected graph G. The edge e is said to be incident on the vertices u and v.
10. A path from a vertex u to a vertex v is a sequence of vertices $u_1, u_2, \ldots, u_n$ such that $u = u_1$, $u_n = v$, and (u_i, u_{i+1}) is an edge for all $i = 1, 2, \ldots, n - 1$.
11. The vertices u and v are called connected if there is a path from u to v.
12. A simple path is a path in which all the vertices, except possibly the first and last vertices, are distinct.
13. A cycle in G is a simple path in which the first and last vertices are the same.
14. An undirected graph G is called connected if there is a path from any vertex to any other vertex.
15. A maximal subset of connected vertices is called a component of G.
16. Suppose that u and v are vertices in a directed graph G. If there is an edge from u to v, that is, $(u, v) \in E$, we say that u is adjacent to v and v is adjacent from u.
17. A directed graph G is called strongly connected if any two vertices in G are connected.
18. Let G be a graph with n vertices, where $n > 0$. Let $V(G) = \{v_1, v_2, \ldots, v_n\}$. The adjacency matrix A_G is a two-dimensional $n \times n$ matrix such that the (i, j)th entry of A_G is 1 if there is an edge from v_i to v_j; otherwise, the (i, j)th entry is 0.
19. In an adjacency list representation, corresponding to each vertex v is a linked list such that each node of the linked list contains the vertex u and $(v, u) \in E(G)$.
20. The depth-first traversal of a graph is similar to the preorder traversal of a binary tree.
21. The breadth-first traversal of a graph is similar to the level-by-level traversal of a binary tree.
22. The shortest path algorithm gives the shortest distance for a given node to every other node in the graph.
23. In a weighted graph, every edge has a nonnegative weight.

24. The weight of the path P is the sum of the weights of all the edges on the path P, which is also called the weight of v from u via P.
25. A (free) tree T is a simple graph such that if u and v are two vertices in T, there is a unique path from u to v.
26. A tree in which a particular vertex is designated as a root is called a rooted tree.
27. Suppose T is a tree. If a weight is assigned to the edges in T, T is called a weighted tree.
28. If T is a weighted tree, the weight of T, denoted by $W(T)$, is the sum of the weights of all the edges in T.
29. A tree T is called a spanning tree of graph G if T is a subgraph of G such that $V(T) = V(G)$—that is, if all the vertices of G are in T.
30. Let G be a graph and $V(G) = \{v_1, v_2, \ldots, v_n\}$, where $n \geq 0$. A topological ordering of $V(G)$ is a linear ordering $v_{i1}, v_{i2}, \ldots, v_{in}$ of the vertices such that if v_{ij} is a predecessor of v_{ik}, $j \neq k$, $1 \leq j, k \leq n$, then v_{ij} precedes v_{ik}, that is, $j < k$ in this linear ordering.
31. A circuit is a path of nonzero length from a vertex u to u with no repeated edges.
32. A circuit in a graph that includes all the edges of the graph is called an Euler circuit.
33. A graph G is said to be Eulerian if either G is a trivial graph or G has an Euler circuit.

EXERCISES

Use the graph in Figure 12-27 for Exercises 1 through 4.

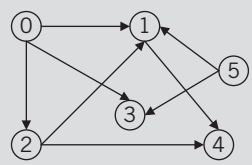


FIGURE 12-27 Graph for Exercises 1 through 4

1. Find the adjacency matrix of the graph.
2. Draw the adjacency list of the graph.
3. List the nodes of the graph in a depth-first traversal.
4. List the nodes of the graph in a breadth-first traversal.

5. Find the weight matrix of the graph in Figure 12-28.

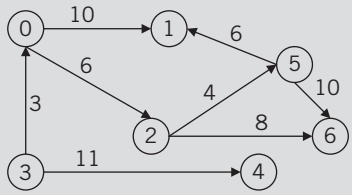


FIGURE 12-28 Graph for Exercise 5

6. Consider the graph in Figure 12-29. Find the shortest distance from node 0 to every other node in the graph.

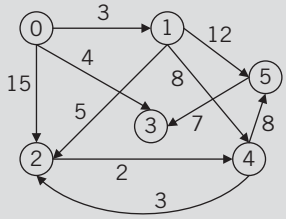


FIGURE 12-29 Graph for Exercise 6

7. Find a spanning tree in the graph in Figure 12-30.

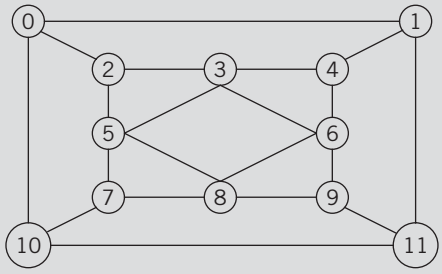


FIGURE 12-30 Graph for Exercise 7

8. Find a spanning tree in the graph in Figure 12-31.

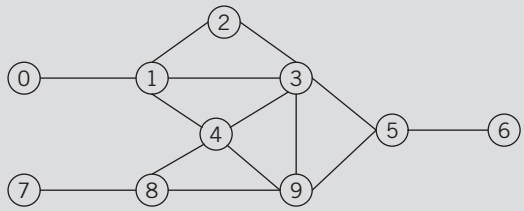


FIGURE 12-31 Graph for Exercise 8

9. Find the minimum spanning tree for the graph in Figure 12-32 using the algorithm given in this chapter.

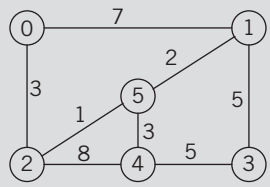


FIGURE 12-32 Graph for Exercise 9

10. List the nodes of the graph of Figure 12-33 in a breadth-first topological ordering.

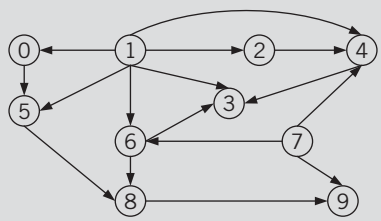


FIGURE 12-33 Graph for Exercise 10

11. Describe whether the graph in Figure 12-34 has an Euler circuit. If the graph has an Euler circuit, find one such circuit.

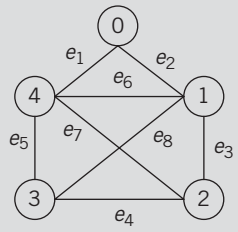


FIGURE 12-34 Graph for Exercise 11

12. Describe whether the graph in Figure 12-35 has an Euler circuit. If the graph has an Euler circuit, find one such circuit.

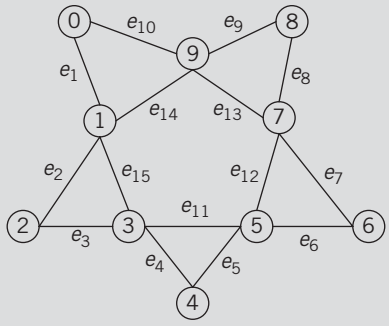


FIGURE 12-35 Graph for Exercise 12

PROGRAMMING EXERCISES

1. Write a program that outputs the nodes of a graph in a depth-first traversal.
2. Write a program that outputs the nodes of a graph in a breadth-first traversal.
3. Write a program that outputs the shortest distance from a given node to every other node in the graph.
4. Write a program that outputs the minimum spanning tree for a given graph.
5. The algorithm to determine the minimum spanning tree given in this chapter is of the order $O(n^3)$. The following is an alternative to Prim's algorithm that is of the order $O(n^2)$.

Input: A connected weighted graph $G = (V, E)$ of n vertices, numbered 0, 1, . . ., $n - 1$; starting with vertex s, with a weight matrix of W.

Output: The minimum spanning tree.

```
Prim2(G, W, n, s)
Let T = (V, E), where E = ϕ.
for(j = 0; j < n; j++)
{
     edgeWeight[j] = W(s,j);
     edges[j] = s;
     visited[s] = false;
}
edgeWeight[s] = 0;
visited[s] = true;
while(not all nodes are visited)
{
     Choose the node that is not visited and has the smallest weight, and
     call it k.
     visited[k] = true;
     E = E ∪ {(k, edges[k])}
     V = V ∪ {k}
     for each node j that is not visited
          if(W(k,j) < edgeWeight[k])
          {
             edgeWeight[k] = W(k,j);
             edges[j] = k;
          }
}
return T.
```

Write a definition of the function `Prim2` to implement this algorithm, and also add this function to the `class msTreeType`. Furthermore, write a program to test this version of Prim's algorithm.

6. Write a program to test the breadth-first topological ordering algorithm.

7. Let G be a graph and $V(G) = \{v_1, v_2, \ldots, v_n\}$, where $n \geq 0$. Recall that a topological ordering of $V(G)$ is a linear ordering $v_{i1}, v_{i2}, \ldots, v_{in}$ of the vertices such that if v_{ij} is a predecessor of v_{ik}, $j \neq k$, $1 \leq j \leq n$, $1 \leq k \leq n$, then v_{ij} precedes v_{ik}, that is, $j < k$ in this linear ordering. Suppose that G has no cycles. The following algorithm, a depth-first topological ordering, lists the nodes of the graph in a topological ordering.

   In a depth-first topological ordering, we start with finding a vertex that has no successors (such a vertex exists because the graph has no cycles), and place it last in the topological order. After we have placed all the successors of a vertex in topological order, we place the vertex in the topological order

before any of its successors. Clearly, in the depth-first topological ordering, first we find the vertex to be placed in `topologicalOrder[n-1]`, then `topologicalOrder[n-2]`, and so on.

Write the definitions of the C++ functions to implement the depth-first topological ordering. Add these functions to the `class topologicalOrderType`, which is derived from the `class graphType`. Also, write a program to test your depth-first topological ordering.

8. Write a program to implement Fleury's algorithm as described in this chapter.

This page intentionally left blank